

Kestrel Cyber MSSP — Platform Operations Bible

Document version: 5.0 (ops bible + journey + chapters 1–17 operator addenda) Verified against live systems:
2026-07-31 Control plane: VM 100 — hostname mssp-control — IP 192.168.0.201 Repository on disk:
/opt/mssp-control

Document identity

Field	Value
Product name	Kestrel Cyber MSSP Control Plane
What this document is	The complete, self-explanatory operations bible: the full journey story, every server, every open-source engine and how they wire together, every major role, data paths, day-2 commands, and the decisions we finally locked in
What this document is not	Marketing copy, a wish-list of future tools, or a paste of failed terminal logs
Markdown source	/opt/mssp-control/docs/MSSP_PLATFORM_MASTER_BLUEPRINT.md
PDF export	/opt/mssp-control/DOCS/MSSP_PLATFORM_MASTER_BLUEPRINT.pdf
Word export	/opt/mssp-control/DOCS/MSSP_PLATFORM_MASTER_BLUEPRINT.docx
Regenerate exports	See Appendix R at the end of this book

Truth order when documents disagree:

1. What is running on the live VMs / database right now 2. This Operations Bible (v3) 3. Git commits and validation scripts that print PASS 4. Older Knowledge Base (KB) markdown files

How to use this book

Read it like a field manual, not like an academic paper.

If you need...	Go to...
“Tell me the story from Day 1 — challenges, fixes, decisions”	Part S (start here for narrative)
“What did we build and why?”	Part A
“Which VM is which IP / which tool?”	Part B
“Who can log in where? What is customer_admin?”	Part C
“How does an alert get from a PC to my screen?”	Part D
“What does Isolate actually do? Why did it fail before?”	Part E
“Give me the exact commands”	Part F
“What screens and database tables exist?”	Part G
“What must we not oversell?”	Part H
“Compressed timeline”	Part I
“Open-source tools, jobs, and wiring map”	Part S §§5–S.6 (also summarized in B)
“Where are the files on disk?”	Appendix S

If you need...	Go to...
Troubleshooting when something breaks	Addendum 1
Day-in-the-life walkthroughs	Addendum 2
Ports / firewall between VMs	Addendum 3
Secrets & who can access what	Addendum 4
Backup & recovery	Addendum 5
Printable checklists	Addendum 6
Glossary	Addendum 7
Service catalog (what we sell)	Addendum 8
Customer-safe vs SOC-only data	Addendum 9
Audit / isolate accountability story	Addendum 10
KB document index	Addendum 11
Validator script index	Addendum 12
Known quirks	Addendum 13
Bible version changelog	Addendum 14
UI tour (Admin + Customer)	Addendum 15
E2E evidence samples	Addendum 16
Platform incident response plan	Addendum 17

Parts A–I = field manual. Part S = journey story. Addenda 1–17 = runbooks, catalogs, indexes, and IR.

Part S — The structured journey story (how it all started, what broke, what we decided)

This chapter is the narrative spine of the platform. Parts A–I are the operating manual. Part S is the story + decisions + tool wiring in one place so a future operator (or a future you) can understand **why** the system looks the way it does.

S.1 How it all started — the founding decision

The original problem

You did not want “another Wazuh login for customers.” You wanted a real MSSP product:

- Your SOC team works in one branded Admin portal.
- Each paying customer works in their own branded Customer portal.
- Multiple customers share the same backend without ever seeing each other’s data.
- Detection engines (Wazuh and friends) stay in the back room as adapters — never as the customer-facing UI.
- The same design must work on local servers now and migrate to cloud later without inventing a second product.

The founding architecture (locked early and never silently changed)

Decision	Final choice
Product brain	FastAPI + PostgreSQL + Redis on VM 100
SOC UI	frontend-admin on port 3000 (nginx production build)
Customer UI	frontend-customer on port 3001 (nginx production build)
Engines	Open-source SOC stack behind adapters
Tenant isolation	Customer wrong-tenant → HTTP 404 (not 403)
Customer UI calling Admin APIs	Forbidden forever
Streamlit as the product	Rejected (prototypes only, never customer-facing)
Lab wording in dashboards / fail-open DEMO tenant	Rejected for production path

Those rules live permanently in AGENTS.md, CLAUDE.md, and .cursor/rules/mssp-control-plane.mdc.

The working method that shaped everything

From KB-009A onward, the project used a strict AI/human workflow:

1. Inspect live files and git — do not trust memory. 2. Plan and stop for approval on real decisions. 3. Implement only approved scope. 4. Validate with a module script until PASS. 5. Commit / tag / snapshot only when the human asks.

That discipline is why the repository has dozens of scripts/kb0NN_validate_*.sh files and an AI_PROMPT_LEDGER.md.

S.2 The build story — chapter by chapter

Chapter 1 — Foundation (roughly KB-001 → KB-009A)

What we achieved:

- Docker Compose stack on VM 100 (mssp-postgres, mssp-redis, mssp-backend-api, later both frontends).
- Core PostgreSQL schema (tenants, platform_users, alerts, incidents, appliances, audit, ...).
- Permanent agent rules so every future coding session inherits the same security and architecture constraints.

Chapter 2 — Identity and walls (KB-010 → KB-017)

What we achieved:

- Login + JWT + bcrypt.
- Five roles: platform_admin, soc_manager, soc_analyst, customer_admin, customer_viewer.
- Protection of Admin and Customer APIs.
- Tenant management, user management, appliance activation tokens, appliance register/heartbeat, credential rotation.

Key human decisions (examples that still govern today):

- Only platform_admin creates/edits tenants and many high-risk admin writes; SOC often read-only on those surfaces.
- Staff may read /customer/* for support; customers never read /admin/*.
- Soft-disable users/appliances instead of hard-delete (preserve audit/history).

- Activation-token failures all look the same (401) — no oracle that reveals “expired vs revoked.”

Chapter 3 — Two portals become a product (KB-018 → KB-035)

What we achieved:

- Admin portal foundation and activation-token UI.
- Full Customer portal: dashboard, alerts, incidents, assets, appliances, reports, recommendations, notifications, account.
- Validated baseline tagged through kb035-customer-appliance-detail-validated.

Business meaning: customers finally had a real product surface — not a prototype dump of engine screens.

Chapter 4 — Enterprise roadmap, then real iron (KB-036 → KB-060 planning; then live installs)

KB-036 locked the MSSP / SOC / MDR / XDR roadmap and deployment models (cloud / on-prem / hybrid).

Then the project stopped being “docs about tools” and became “tools on VMs”:

Order	What went live	VM / IP
1	Ansible controller	VM 112 / 192.168.0.222
2	Wazuh 4.14.6 Manager/Indexer/Dashboard	VM 101 / 192.168.0.211
3	Linux agent proof (later decommissioned host)	VM 105 (destroyed 2026-07-29)
4	Suricata IDS + Wazuh agent forward	VM 106 / 192.168.0.216
5	TheHive + Shuffle co-located	VM 102 / 192.168.0.212
6	Greenbone CE + later Nuclei+Vuls	VM 109 / 192.168.0.219
7	Windows endpoint + telemetry + AR	VM 104 / 192.168.0.214

Important correction decided with you: on-prem local servers are the production MSSP path now, not a disposable lab. Same architecture migrates to cloud later.

Chapter 5 — Wiring the SOC loop (KB-049, KB-061–063, later sync)

What we achieved:

- Wazuh can raise work into Shuffle → TheHive (auto-ticket path).
- Control plane can sync SOC objects (POST /integrations/soc/sync).
- Instant Wazuh ingress into the control plane so portals are not waiting on slow detours.
- Admin triage, recommendations, notifications, monthly PDF/Excel reports.

Chapter 6 — Entitlements, vulns, production posture (KB-068–079, posture hardening)

What we achieved:

- Vuln findings land in vulnerabilities and can become customer recommendations.
- Decision: Nuclei + Vuls are the \$0 primary scanners; Greenbone CE is co-located backup; Greenbone Enterprise is deferred (KB-077).
- Decision: scanners must not live on the control plane — after a mistaken early install path, Nuclei+Vuls were moved to VM 109.

- Production posture: nginx builds, no lab/demo dashboard wording, fail-closed Wazuh tenant mapping, THEHIVE_DEFAULT_ORG=MSSP.

Chapter 7 — Co-managed EDR / MXDR (KB-083–091)

What we achieved:

- Isolate / unisolate / kill / forensics / block-hash APIs and UI on Admin + Customer (role-gated).
- Process trees, forensic streaming uploads, EDR sweeper, Windows AR pack.
- Containment honesty register (KB-091): never claim success without endpoint-proven effect.
- Audit enrichment so co-managed isolate shows who / when / which portal / which agent.
- List search + pagination and dashboard KPI link corrections.

Chapter 8 — This Operations Bible

You asked for a single self-explanatory master document. Versions evolved:

- v1 — too dense / some mismatches
- v2–v3 — operator rewrite + full VM catalog + commands
- v4 (this) — adds the full journey story, challenge/fix catalog, open-source tool jobs, and wiring map

S.3 Challenges, errors, and how we sorted them out

This is not a list of typos. These are the material engineering and product problems that changed the design.

Challenge 1 — “Just expose Wazuh / Streamlit to customers”

Item	Detail
Risk	Raw logs, broken multi-tenancy, no product brand
Resolution	Dual portals + normalization + customer-safe field stripping + customer_safe_labels.py

Challenge 2 — Password leaked in HTTP 422 validation errors (KB-014)

Item	Detail
Symptom	FastAPI default validator echoed plaintext password inside error input
Resolution	Global sanitizing RequestValidationError handler (backend-api/app/core/error_handlers.py) redacting sensitive keys

Challenge 3 — Validation scripts lying / false failures

Item	Detail
Examples	KB-015 psql INSERT 0 1 glued onto UUID; KB-017 content-grep false positive
Resolution	Harden scripts (psql_scalar, UUID checks, git diff --quiet) — treat validators as production code

Challenge 4 — Shuffle blocked auto-ticket wiring

Item	Detail
Symptom	Shuffle health said admin user must be set up first
Resolution	Complete first-login/admin setup on VM 102, then finish KB-049 live wiring (Wazuh → Shuffle → TheHive proof)

Challenge 5 — Scanners on the control plane (wrong blast radius)

Item	Detail
Mistake	Early Nuclei+Vuls path touched VM 100
Your correction	Keep control plane free of scanner engines
Resolution	Install /opt/mssp-vuln-free on VM 109; control plane only syncs findings

Challenge 6 — Lab vs production confusion

Item	Detail
Tension	Early DEMO tenants / lab org names vs enterprise MSSP goal
Resolution	Explicit posture: on-prem = production path; fail-closed tenant mapping; strip lab/demo dashboard copy; prefer THEHIVE_DEFAULT_ORG=MSSP (note: some older sync scripts still default MSSP-Lab if env unset — operators must set THEHIVE_ORG)

Challenge 7 — Customer login 502 after API recreate

Item	Detail
Root cause	nginx upstream cached stale Docker IP for backend-api
Resolution	Recreate both frontends with the API; nginx resolver 127.0.0.11

Challenge 8 — Host isolation looked successful but was not

Item	Detail
Symptom	UI showed Isolated/verified; Windows host still reachable (ping/RDP)
Root causes	(a) “Manager accepted AR” + “agent online” treated as proof; (b) Windows .cmd deadlocks/more/JSON-on-argv; (c) OS detector defaulted to Linux; (d) weak firewall semantics; (e) PowerShell Unicode/\$name: parse bugs; (f) Admin UI missing isolate on full incident page
Resolution	OS-aware fail-closed AR; STDIN .cmd wrappers; default-deny quarantine .ps1; Dispatched vs Verified honesty; proof script; Admin+Customer containment UI; audit enrichment; KB-091 gap register

Challenge 9 — Dashboard tiles linked to the wrong places

Item	Detail
Symptom	“Events Collected” opened appliances; “Automation/SLA” opened customers
Resolution	Retarget KPIs (events→alerts, collector health→appliances, recommendations→recommendations); clarify EDR metrics strip is summary-only

Challenge 10 — Role naming confusion (customer_admin)

Item	Detail
Symptom	Document/UI mismatch made it seem the role was removed
Live truth	Role still exists; UI label is Administrator; DB/API name is customer_admin (admin@alphawin.com active)
Resolution	Operations Bible documents both names side by side

Challenge 11 — Linux lab endpoint disappeared

Item	Detail
Event	VM 105 destroyed 2026-07-29
Resolution	Document decommission; Windows path (VM 104) became the live containment proof host; Linux reinstall guide retained

Challenge 12 — Database pressure under concurrent load

Item	Detail
Problem	Single-connection patterns starved under portals + sync + EDR
Resolution	psycopg_pool.ConnectionPool in backend-api/app/db/session.py

Challenge 13 — Large forensic uploads risking OOM

Item	Detail
Problem	Buffering whole ZIP in memory / SOAR body
Resolution	HMAC upload URLs + request.stream() chunked write (edr_forensics_storage.py)

Challenge 14 — EDR actions stuck forever in EXECUTING

Item	Detail
Resolution	Background edr_sweeper.py transitions timed-out executions

S.4 Decisions we locked at the end (the “constitution”)

These are the standing agreements after the journey above. Do not silently reverse them.

1. Control plane is the product. Engines are adapters. 2. On-prem now = production MSSP path. Cloud is the same design later. 3. Two portals, portal-separated login. Staff on :3000, customers on :3001. 4. Five roles remain including customer_admin / customer_viewer (UI: Administrator / Viewer). 5. Customer wrong-tenant → 404. No enumeration. 6. Customer never sees raw engine dumps or third-party product names in portal copy. 7. Fail-closed tenant mapping for alerts (no DEMO default). 8. Vuln primary stack = Nuclei + Vuls on VM 109; Greenbone CE backup; Enterprise deferred. 9. No scanners on VM 100. 10. Containment honesty: Dispatched ≠ Verified; agent-online is not isolation proof. 11. Co-managed EDR allowed for Customer Administrators; viewers are read-only; audit must show accountability. 12. Validation before commit; humans decide commits/tags/snapshots. 13. Zeek / MISP / Velociraptor / monitoring VM remain future unless an approved KB installs them. 14. Rebuild both frontends whenever backend-api is recreated (502 prevention).

S.5 Open-source backend tools — what each one is for, and what we achieved with it

Tool	Where it runs	Job in the MSSP	What we achieved with it
PostgreSQL 16	VM 100 Docker	System of record for tenants, users, alerts, incidents, EDR, audit, vulns	Durable multi-tenant product data
Redis 7	VM 100 Docker	Cache / supporting queue patterns	Health-checked dependency for API
FastAPI (Python)	VM 100 Docker	Product API, auth, adapters, EDR orchestration	The only API portals talk to
React + nginx	VM 100 Docker	Admin + Customer UIs	Production static portals on 3000/3001
Docker Compose	VM 100	Orchestrate control-plane containers	Repeatable product runtime
Wazuh 4.14.6	VM 101	SIEM, agent management, rules, Active Response	Endpoint/sensor telemetry + isolate/kill dispatch
Wazuh agents	Endpoints + sensors	Collect OS/network logs and execute AR	Windows/Linux/Suricata host coverage
Suricata	VM 106	Network IDS	Network detections forwarded via Wazuh agent
TheHive	VM 102	Case / IR ticket system	Analyst cases linked to automation
Shuffle	VM 102	SOAR playbooks / webhooks	Wazuh→case automation and control-plane hops
Nuclei	VM 109 /opt/mssp-vuln-free	Template vuln scanning	Primary free vuln coverage into control plane

Tool	Where it runs	Job in the MSSP	What we achieved with it
Vuls	VM 109 /opt/mssp-vuln-free	Host/package CVE scanning	Primary free host CVE path
Greenbone CE (OpenVAS)	VM 109	Classic NVT scanning + GSA UI	Backup vuln engine; Enterprise deferred
Sysmon	Windows endpoints	Process/network telemetry	Process trees / EDR visibility
Ansible	VM 112	Remote install/config automation	Safe-default installs for engines
Linux firewall / Windows Firewall (netsh)	Endpoints via AR scripts	Host quarantine	Real containment when scripts apply

Optional / not confirmed live as product engines yet: Zeek, MISP, Velociraptor, Prometheus/Grafana.

S.6 How the tools are wired between each other

Wiring diagram (logical)

```

[Windows/Linux endpoints]
  | Wazuh agent (+ Sysmon on Windows)
  v
[Wazuh Manager VM 101]
  |-- Active Response --> endpoint isolate/kill scripts
  |-- Instant ingress hook --> Control Plane API (VM 100)
  |-- Integration / webhook --> Shuffle (VM 102)
  |
  | v
  | TheHive cases (VM 102)
  |
  | v
  | SOC sync pull/push --> Control Plane
  |
[Suricata VM 106] --eve.json--> [Wazuh agent] --> [Wazuh Manager] --> (same as above)

[Nuclei / Vuls / Greenbone VM 109]
  |
  | pull scripts / instant hook
  |
  v
[Control Plane vuln sync API] --> vulnerabilities --> Admin triage
                                --> customer recommendations

[Control Plane PostgreSQL] <--> Admin portal :3000
                                <--> Customer portal :3001

[Ansible VM 112] --SSH--> installs/configures 101/102/106/109/...
```

Wiring in plain English (seven pipes)

1. Agent pipe: Endpoint → Wazuh Manager (telemetry + AR execution channel). 2. IDS pipe: Suricata → local Wazuh agent → Manager. 3. Instant product pipe: Manager/integrations → control plane /integrations/... so portals update quickly. 4. SOAR/case pipe: Manager → Shuffle → TheHive; control plane can also sync cases/alerts. 5. Vuln pipe: Scanners on 109 → pull/sync → vulnerabilities → SOC → customer-safe recommendations. 6. Containment pipe: Portal button → control plane EDR API → Wazuh Active Response → endpoint firewall/script → audit log. 7. Automation pipe: Ansible controller configures the engine VMs so installs are repeatable and gated.

What is intentionally **not** wired to customers

- Direct Wazuh Dashboard logins
- Direct TheHive/Shuffle/Greenbone admin UIs as the product

- Raw eve.json, raw_event JSON, packet captures, internal notes
-

S.7 What “done enough to operate” means today (honest ending of the story)

You can run a real multi-tenant MSSP loop today:

- Onboard a tenant and Customer Administrator.
- Enroll Windows agents with telemetry.
- See alerts/incidents in both portals.
- Pull vulns from the free stack.
- Dispatch isolate/kill with audit accountability.
- Export monthly reports.

You must still be honest about unfinished edges (KB-091): automatic Verified callbacks, true hash blocking, Zeek/MISP/Velociraptor, Greenbone Enterprise, and rebuilding a Linux lab endpoint when needed.

That honesty is part of the product story — not a footnote.

Part A — Big picture (expanded)

A.1 What business problem this platform solves

You are building a Managed Security Service Provider (MSSP) business. That means:

- You protect multiple customer organizations (tenants) from one SOC team.
- Each customer must see only their own security picture.
- Your SOC must see across customers when their role allows it.
- Customers must receive plain-English summaries and actions — not raw SIEM dumps.
- You must be able to contain a compromised endpoint (isolate / kill process) from *your* product UI, not by SSHing into every laptop manually.

The software product that delivers that is the Kestrel Cyber MSSP Control Plane on VM 100.

A.2 What the product is (in one paragraph)

The control plane is a dual-portal web product. Admin portal (<http://192.168.0.201:3000>) is for your MSSP / SOC staff. Customer portal (<http://192.168.0.201:3001>) is for each paying customer's own users. Behind both portals sits one FastAPI backend, one PostgreSQL database, and one Redis instance. Behind *that* sit security engines (Wazuh, Suricata, TheHive, Shuffle, Nuclei, Vulns, Greenbone CE). Those engines feed the control plane through adapters and sync scripts. Customers never receive Wazuh, TheHive, or Greenbone logins as the product experience.

A.3 The three layers (memorize this)

LAYER 1 — HUMAN PRODUCT

Admin portal :3000 Customer portal :3001

LAYER 2 — CONTROL PLANE (VM 100)

FastAPI API :8000

PostgreSQL (system of record)

Redis

LAYER 3 — SECURITY ENGINES (other VMs)

Wazuh 101 | TheHive+Shuffle 102 | Suricata 106

Rule of the architecture: Layer 1 and Layer 2 are “the product.” Layer 3 is “the machinery.” If Layer 3 changes brands tomorrow, Layer 1 should still look like Kestrel Cyber — because we normalize into our own tables.

A.4 Design choices we deliberately rejected

Rejected approach	Why we rejected it	What we do instead
Give each customer a Wazuh dashboard login	Breaks multi-tenant isolation; exposes raw logs; not a branded product	Normalize into PostgreSQL; Customer portal shows safe fields only
Build the whole product in Streamlit	Fine for prototypes; weak for multi-tenant RBAC portals	React + nginx production builds for both portals
Install Nuclei/Vuls/Greenbone on VM 100	Couples scanners to the product host; wrong blast radius	Scanners on VM 109; control plane only syncs results
Fail-open “DEMO” tenant when agent group is missing	Silently mixes customer data	Fail-closed mapping — binding required
Claim “Isolated” because the agent is still online	False confidence (agent <i>*must*</i> stay reachable to Manager)	Prefer Dispatched until endpoint proves quarantine

A.5 Deployment models (business view)

The same control-plane design supports:

Model	Meaning for you
On-prem now (current)	All VMs on your local network — this <i>*is*</i> production for the MSSP path
Cloud later	Same product APIs/portals; move hosts when customer volume justifies it
Hybrid	Some tenants cloud-connected, some on-prem appliances (see KB-073 deployment mode)

You do not maintain two different products. You maintain one architecture that can move.

A.6 Live snapshot of **your** environment (2026-07-31)

This is not theory — it was read from the live database and network the day v3 was written.

Item	Live value
Control plane containers	mssp-postgres, mssp-redis, mssp-backend-api, mssp-frontend-admin, mssp-frontend-customer
Active customer tenant	Alpha-Win-Corp (short code observed as ALPHAWINCORP-6VS2)
Customer Administrator login	admin@alphawin.com — DB role customer_admin — UI label Administrator
MSSP staff logins	platform.admin@example.local, soc.manager@example.local, soc.analyst@example.local

Item	Live value
Approx. stored volume	~344 security_alerts, ~9 incidents, ~7 edr_action_executions
Windows proof endpoint	Hostname WIN-BL72S84GDTF, IP 192.168.0.214 (VM 104)
Linux lab endpoint	VM 105 destroyed 2026-07-29
Engine reachability sample	Wazuh API :55000 up (401 = needs auth), TheHive :9000 up, Shuffle :3001 on VM 102 up, Greenbone GSA :443 up

A.7 What “enterprise-ready” means on this project

Every backend tool, adapter, and security control must be production-grade or carry an explicit dated gap with an upgrade path. Example: Nuclei+Vuls is the approved \$0 primary vuln stack today; Greenbone Enterprise is deferred with a written plan (docs/KB077_GREENBONE_ENTERPRISE_READINESS_PLAN.md). Lab shortcuts (demo tenant defaults, lab wording in dashboards) are not allowed in runtime product surfaces.

Part B — Server catalog (fully elaborated)

This part answers, for every important machine: What is its VM number? Hostname? IP? Why does it exist? What software runs on it? What ports matter? How does the control plane talk to it? What happens if it is down?

B.1 Master map (print this page)

VM	Hostname	IP	Status	One-line purpose
100	mssp-control	192.168.0.201	LIVE	Product: portals + API + DB + Redis
101	wazuh-stack	192.168.0.211	LIVE	SIEM / agents / Active Response brain
102	thehive_shuffle	192.168.0.212	LIVE	Cases (TheHive) + automation (Shuffle)
104	windows-endpoint-lab	192.168.0.214	LIVE test endpoint	Windows agent + Sysmon + quarantine scripts
105	*(removed)*	*(was .215)*	DESTROYED	Former Linux agent lab
106	suricata-sensor	192.168.0.216	LIVE	Network IDS sensor (+ optional Zeek slot)
109	greenbone	192.168.0.219	LIVE	Vuln scanners: Nuclei + Vuls + Greenbone CE
112	automation	192.168.0.222	READY	Ansible controller

Inventory file on disk: /opt/mssp-control/ansible/inventory/hosts.yml

Not live yet (placeholders only)

VM	Hostname	Planned IP	Notes
108	misp	192.168.0.218	Threat intel — pending approved KB install
110	velociraptor	192.168.0.220	DFIR — pending
111	monitoring	192.168.0.221	Prometheus/Grafana — pending
103	separate Shuffle	—	Not used; Shuffle already on VM 102
107	separate Zeek	—	Not used; Zeek would co-locate on VM 106

B.2 VM 100 — Control plane (mssp-control / 192.168.0.201)

Why this VM exists

This is the product host. If VM 100 is down, portals and APIs are down — even if Wazuh is still collecting. Treat it like a production application server, not a disposable lab box.

Software (Docker Compose)

Compose file: /opt/mssp-control/docker-compose.yml Project directory: /opt/mssp-control

Container	Image / build	Published port	Responsibility
mssp-postgres	postgres:16-alpine	none (internal)	System of record
mssp-redis	redis:7-alpine	none (internal)	Cache / supporting queue patterns; password required
mssp-backend-api	build ./backend-api	\${API_PORT}:8000	Auth, RBAC, all business APIs, engine adapters, EDR
mssp-frontend-admin	build ./frontend-admin	3000→80	SOC/Admin SPA (nginx)
mssp-frontend-customer	build ./frontend-customer	3001→80	Customer SPA (nginx)

Docker network name: mssp-backend (bridge). Volumes: postgres_data, redis_data.

Secrets mounting (important)

The API does not hardcode Wazuh/TheHive passwords in source. Compose mounts files from /opt/mssp-control/.secrets/ into /run/secrets/ inside the container (read-only), for example:

- wazuh_api_user, wazuh_api_password
- thehive_password
- soc_sync_api_key, wazuh_ingress_token
- shuffle_webhook_url, vuln_sync_api_key

Never commit .env or .secrets/.

How browsers reach the API

1. User opens `http://192.168.0.201:3000` or `:3001`. 2. React app calls paths like `/api/auth/login`. 3. nginx in the frontend container proxies `/api/` to `http://backend-api:8000/` (strip `/api`). 4. nginx uses Docker DNS resolver `127.0.0.11` so if the API container is recreated with a new IP, the proxy can re-resolve.

Operational lesson: If you rebuild only `backend-api` and leave old frontend containers running with a stale upstream, Customer/Admin login can return HTTP 502. Fix: rebuild/recreate both frontends together with the API.

What “healthy” looks like

```
curl -fsS http://localhost:8000/health | jq .
```

Expect JSON with API, database, and Redis all ok.

If VM 100 is down

Impact	Detail
Portals unreachable	SOC and customers cannot work in product UI
Engines may still collect	Wazuh can still receive agent events
Sync lags	Vuln pulls / TheHive sync cannot land in PostgreSQL until API is back

B.3 VM 101 — Wazuh stack (wazuh-stack / 192.168.0.211)

Why this VM exists

Wazuh is your SIEM / endpoint telemetry / Active Response engine. Agents on endpoints and sensors report here. The control plane asks this Manager to run `isolate/kill` commands.

What runs here

Component	Job in plain English
Wazuh Manager	Receives agent data, evaluates rules, stores alerts, executes Active Response
Wazuh Indexer	Search backend (OpenSearch family) for Wazuh data
Wazuh Dashboard	Engineer console for Wazuh itself — not the customer product
Wazuh API (:55000)	Machine API used by our control plane

Live version noted in project context: Wazuh 4.14.6.

Ports that matter

Port	Meaning
55000/tcp	Wazuh API (<code>https://192.168.0.211:55000</code>) — live probe returned 401 (service up, auth required)
1514/tcp	Agent event channel (standard)
1515/tcp	Agent enrollment (standard)

How the control plane talks to it

Compose defaults:

- `WAZUH_API_URL=https://192.168.0.211:55000`

- Credentials via secret files
- WAZUH_API_VERIFY_TLS often false in current on-prem lab/prod path (TLS verify policy should be hardened as you leave internal networks)

Key code: backend-api/app/services/wazuh_client.py (agent OS lookup, Active Response dispatch).

Tenant mapping (critical)

Every agent should belong to a Wazuh group bound to a tenant (engine bindings / tenant_<SHORTCODE> pattern). WAZUH_DEFAULT_TENANT_SHORT_CODE is empty by design = fail-closed. Missing binding must not silently attach alerts to a demo customer.

If VM 101 is down

- New endpoint telemetry stops arriving.
- Isolate/kill from the portals cannot be dispatched.
- Historical alerts already in PostgreSQL remain visible.

Ansible / SSH

Inventory uses user secadmin and key id_ed25519_ansible_wazuh from the automation controller patterns.

B.4 VM 102 — TheHive + Shuffle (thehive_shuffle / 192.168.0.212)

Why this VM exists

You need:

1. A case / IR ticket system (TheHive).
2. A SOAR / playbook system (Shuffle).

To save hardware, both run co-located on VM 102. Roadmap VM 103 (separate Shuffle) is not deployed.

Tools and ports

Tool	Port on VM 102	Purpose	Live check
TheHive	9000	Cases, observables, analyst IR workflow	HTTP responded
Shuffle	3001	Workflows, webhooks, automation hops	HTTP 200

Port confusion warning (read carefully):

- Customer portal 3001 = on VM 100 (192.168.0.201)
- Shuffle UI 3001 = on VM 102 (192.168.0.212)

Same port number, different machines. Bookmark by IP.

How the control plane talks to TheHive

- THEHIVE_URL=http://192.168.0.212:9000
- THEHIVE_DEFAULT_ORG=MSSP (preferred production default)
- Password via /run/secrets/thehive_password

Sync helper scripts:

- scripts/kb061_sync_thehive_alerts.sh
- scripts/kb061_run_periodic_sync.sh

Operator trap: kb061_sync_thehive_alerts.sh still defaults THEHIVE_ORG to MSSP-Lab if unset. Always export THEHIVE_ORG=MSSP (or your real org) when running sync.

Shuffle's role with the control plane

Shuffle can create cases and call back into the control plane (SOC sync / EDR workflow helpers). Helper script:

- scripts/kb062_shuffle_control_plane_hop_helper.sh

Webhook URL is provided to Compose via secret file shuffle_webhook_url.

If VM 102 is down

- Case automation and TheHive sync pause.
- Core portals and PostgreSQL alerts/incidents can still function for already-ingested data.
- Some EDR “notify Shuffle” paths degrade (Wazuh AR may still dispatch independently).

B.5 VM 104 — Windows endpoint lab (windows-endpoint-lab / 192.168.0.214)

Why this VM exists

This is not a scanner and not a portal. It is a real Windows endpoint used to prove:

- Agent enrollment and telemetry
- Process tree visibility (Sysmon / 4688)
- Host isolation / unisolation
- Kill process Active Response

Live identity

Field	Value
Inventory hostname	windows-endpoint-lab
Observed OS hostname	WIN-BL72S84GDTF
IP	192.168.0.214
Role	Wazuh agent Windows + MSSP AR pack
Customer association	Alpha-Win-Corp tenant path

Software that must be present for “EDR-ready”

Component	Why required
Wazuh agent	Ships logs/events to Manager 192.168.0.211
Sysmon (baseline XML)	Process create (ID 1), network (ID 3) for process trees
Security auditing 4688 + command line	Native process create fallback
MSSP AR scripts (.cmd/.ps1)	Isolate / kill / block-hash execution

Bootstrap materials live under:

- deploy/windows-endpoint-telemetry/
- scripts/bootstrap_windows_telemetry.ps1
- deploy/wazuh-active-response/windows/

Important: A green Wazuh agent alone is not enough for process-EDR views. Without Sysmon/4688 channels, process trees look empty.

If this host is down

Only this test endpoint loses coverage. Other tenants/agents are unaffected.

B.6 VM 105 — Linux endpoint lab — decommissioned

Field	Value
Former IP	192.168.0.215
Status	Destroyed in Proxmox on 2026-07-29
Docs for rebuild	docs/VM105_MANUAL_REINSTALL.md
Inventory	Commented out in ansible/inventory/hosts.yml

Do not trust older notes that still map agent 001 / BETALINUX to a live host — that mapping is historical.

B.7 VM 106 — Suricata sensor (suricata-sensor / 192.168.0.216)

Why this VM exists

Endpoint agents see host activity. Suricata sees network activity (IDS). Together they give SOC a fuller picture.

What runs here

Component	Purpose
Suricata	Passive intrusion detection on a capture interface
Wazuh agent	Forwards Suricata alerts/logs to Manager VM 101

Typical design: management NIC on the LAN + separate capture bridge for mirrored traffic.

Zeek note (expanded)

Zeek is a network visibility/analysis engine. Your repo contains install/configure scripts aimed at co-locating Zeek on VM 106 (not a separate VM 107). As of 2026-07-31, Zeek was not confirmed active. Until you verify on the host, document Zeek as optional / not live.

If VM 106 is down

Network IDS alerts stop. Endpoint alerts from agents on other hosts continue.

B.8 VM 109 — Vulnerability stack (greenbone / 192.168.0.219)

Why this VM exists

Vulnerability scanning is heavy and noisy. It belongs on a dedicated scanner host, never on the control plane.

Tools on this one VM

Tool	Commercial cost	Role in *your* product
Nuclei	Free	Primary scanner (templates) under /opt/mssp-vuln-free
Vuls	Free	Primary scanner (host/package CVE style) under /opt/mssp-vuln-free
Greenbone Community Edition	Free	Classic NVT backup scanner; GSA UI on HTTPS
Greenbone Enterprise	Paid	Deferred until ~customer volume — KB-077

Live check: Greenbone GSA HTTPS on :443 responded.

How findings reach the product

1. Scans run on VM 109. 2. Pull scripts on VM 100 SSH/query and normalize. 3. Results POST into control plane vuln sync API. 4. SOC triages in Admin portal. 5. Customers see safe recommendations — not raw scanner JSON and not third-party product branding.

Scripts (see Part F for full commands):

- scripts/kb079_pull_nuclei_findings.sh
- scripts/kb079_pull_vuls_findings.sh
- scripts/kb070_pull_greenbone_findings.sh
- scripts/kb079_run_vuln_scans.sh

If VM 109 is down

New vuln ingestion stops. Existing vulnerabilities rows remain in PostgreSQL.

B.9 VM 112 — Automation controller (automation / 192.168.0.222)

Why this VM exists

Repeatable remote installation and configuration via Ansible, so you are not hand-editing every engine VM forever.

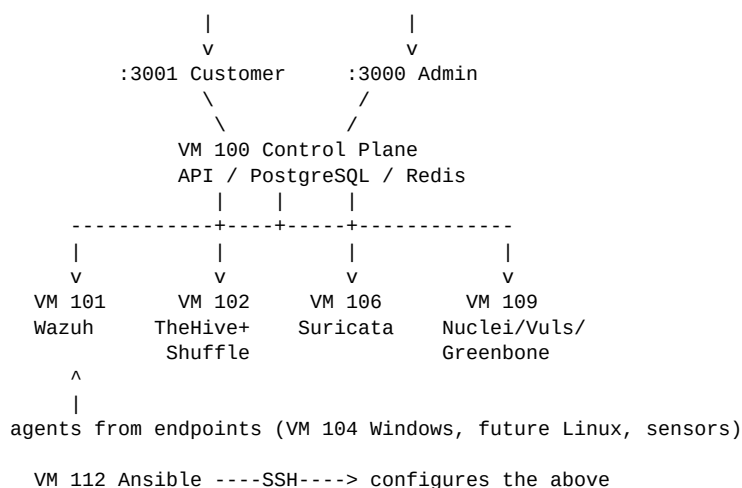
Item	Value
Connection	Often ansible_connection: local on the controller itself
Targets	Other VMs over SSH
Inventory	/opt/mssp-control/ansible/inventory/hosts.yml
Keys	~/.ssh/id_ed25519_ansible_* patterns per host group

If VM 112 is down

Day-2 product traffic can continue (portals/engines). New automated provisioning and playbook-driven changes pause until the controller is back.

B.10 How the VMs connect (relationship diagram)

[Customers browsers]



Part C — People, portals, and roles (fully elaborated)

C.1 Why two portals exist

Portal	Audience	Mental model
Admin :3000	Your employees (platform + SOC)	"We run the MSSP"
Customer :3001	The customer's employees	"We see our security posture and actions"

If you merged them into one UI, you would constantly risk leaking SOC internals and cross-tenant data. Separation is a security control, not a cosmetic choice.

C.2 Portal-separated login (how it works)

When a user logs in, the frontend sends portal=admin or portal=customer with the password.

Backend (backend-api/app/api/routes/auth.py):

- Admin portal accepts only: platform_admin, soc_manager, soc_analyst
- Customer portal accepts only: customer_admin, customer_viewer

Wrong portal → HTTP 403 with a clear message (customer account told to use :3001, staff told to use :3000).

JWT is issued after success (HS256, JWT_SECRET from env). Subsequent API calls send the bearer token.

C.3 The customer_admin clarification (expanded)

This caused confusion in earlier blueprint drafts.

Fact checked in live PostgreSQL on 2026-07-31:

- Role value customer_admin exists in the database constraint.
- User admin@alphawin.com is customer_admin and active.
- The Customer portal Users page labels this role "Administrator" in the dropdown.
- Viewer is labeled "Viewer (read-only)" and stores as customer_viewer.

So:

What you see in the UI	What engineers / DB / APIs call it
Administrator	customer_admin

What you see in the UI	What engineers / DB / APIs call it
Viewer (read-only)	customer_viewer

Nothing in the live code removed customer_admin. If a document says “we no longer have customer_admin,” that document is wrong unless a future migration explicitly drops it (none has).

C.4 Full role encyclopedia

Platform administrator — platform_admin

Topic	Detail
Portal	Admin :3000 only
Scope	Entire platform, all tenants
Typical duties	Create tenants/customers, create staff users, activation tokens, high-risk config
Cannot	Log into Customer portal with this role

SOC manager — soc_manager

Topic	Detail
Portal	Admin :3000
Scope	Cross-tenant SOC
Typical duties	Triage ownership, recommendations, many write ops, vuln promotions
EDR	Allowed to execute containment actions

SOC analyst — soc_analyst

Topic	Detail
Portal	Admin :3000
Scope	Cross-tenant triage
Typical duties	Investigate alerts/incidents, run isolate/kill from Admin
Limits	Some platform-admin-only screens remain read-only / denied

Customer administrator — customer_admin (UI: Administrator)

Topic	Detail
Portal	Customer :3001 only
Scope	One tenant only
Typical duties	Manage that org's users, view incidents/alerts, co-managed EDR when policy allows
Isolation	Any attempt to access another tenant's IDs → 404

Customer viewer — customer_viewer (UI: Viewer)

Topic	Detail
Portal	Customer :3001
Scope	One tenant, read-only
EDR	Blocked from isolate/kill writes
Live count 2026-07-31	Zero users of this role provisioned (role still valid)

C.5 Live user directory (2026-07-31)

Email	Technical role	Portal
platform.admin@example.local	platform_admin	Admin
soc.manager@example.local	soc_manager	Admin
soc.analyst@example.local	soc_analyst	Admin
admin@alphawin.com	customer_admin	Customer

C.6 Co-managed security model

“Co-managed” means:

- MSSP SOC can act on a tenant's incidents.
- The customer's Administrator can also take certain endpoint actions (isolate/unisolate/kill/forensics/block-hash) for their tenant.
- Every action should land in audit_logs with actor, role, portal, source IP, target agent, incident linkage.

This is why audit detail pages and search were added to both portals.

C.7 Where roles are enforced in code (for auditors)

Layer	Location
Database CHECK	postgres/init/002_kb010_auth_rbac.sql
Login portal gate	backend-api/app/api/routes/auth.py
JWT + role helpers	backend-api/app/core/security.py, api/dependencies.py
Customer tenant match	require_tenant_match() → 404
EDR role sets	backend-api/app/services/edr_actions.py (SOC_WRITE_ROLES, CUSTOMER_ACTION_ROLES, READ_ONLY_CUSTOMER)
Admin user UI	frontend-admin/src/pages/UsersPage.tsx
Customer user UI	frontend-customer/src/pages/UsersPage.tsx

Part D — Data journey (fully elaborated)

D.1 Why this section matters

If you only know screens, outages feel random. If you know the journey, you can ask: “Did the agent see it? Did Manager rule it? Did sync land in Postgres? Did the API filter it from the customer?”

D.2 Journey A — Endpoint process event → Admin process tree

1. User or malware starts a process on WIN-BL72S84GDTF. 2. Sysmon Event ID 1 (and/or Security 4688) records process create + command line. 3. Wazuh agent ships the log channel to Manager 192.168.0.211. 4. Manager parses/rules; event may become a Wazuh alert and/or enrich process telemetry. 5. Control plane ingress/sync path writes normalized rows (security_alerts and/or edr_process_events). 6. Admin UI requests /v1/edr/telemetry/process-tree for an incident/host. 7. Widget renders parent/child process relationships (GUID lineage preferred, PID fallback).

Breakpoints if empty tree: Sysmon missing, ossec.conf localfile missing, wrong agent, sync not running, tenant filter mismatch.

D.3 Journey B — Suricata network alert → portals

1. Suricata on VM 106 matches a signature on the capture interface. 2. Alert written to eve.json (typical Suricata output). 3. Local Wazuh agent forwards to Manager. 4. Manager rules → alert. 5. Control plane ingest → security_alerts with tenant mapping from sensor/agent binding. 6. Admin sees technical triage fields; Customer sees safe summary if customer_visible rules allow.

D.4 Journey C — Vuln scan → recommendation

1. Nuclei or Vuls (or Greenbone) runs on VM 109 against targets. 2. Operator runs pull script on VM 100. 3. Script authenticates to vuln sync API with key from secrets. 4. Rows land in vulnerabilities. 5. SOC reviews in Admin Vulnerabilities page; may promote / create customer recommendations. 6. Customer sees action items without scanner brand names.

D.5 Journey D — Alert → incident → TheHive/Shuffle (high level)

1. High-signal alert arrives in control plane. 2. SOC (or automation) creates/links an incidents row. 3. Shuffle playbook may create/update a TheHive case. 4. Periodic TheHive sync can enrich control-plane visibility. 5. Customer sees incident summaries approved for them — not internal notes.

D.6 Normalization contract

Regardless of source engine, the control plane aims to store tenant-scoped business records with fields conceptually like:

tenant, source_platform, asset, alert, incident/case, recommendation, vulnerability, report, visibility_status, sync_health_status

Customer APIs strip forbidden fields (raw_event, internal_notes, many IPs, secrets, etc.).

Part E — Endpoint response / EDR (fully elaborated)

E.1 Product meaning of isolate

When an analyst clicks Isolate host, the intended real-world effect is:

- Host cannot talk freely to the LAN or Internet (lateral movement and C2 blocked).
- Host can still talk to Wazuh Manager (so you can observe and un-isolate).
- DHCP/loopback remain available so the machine does not strand itself uselessly.

On Windows this is implemented primarily with Windows Firewall profile policy via netsh advfirewall (default outbound block, inbound hardening, explicit allows for Manager, blocks for RDP/SMB/WinRM/SSH patterns).

E.2 Where operators click

Place	Portal	Notes
Incident detail page	Admin	Full containment controls
Incident side panel	Admin	Faster triage path
Incident views	Customer	Customer Administrator only
EDR metrics strip on dashboard	Both	Counters only — not the action console

E.3 End-to-end isolate sequence

1. User clicks Isolate on an incident tied to an agent id. 2. API checks role + tenant (assert_can_execute_action). 3. Row inserted into edr_action_executions (status progresses toward executing). 4. _resolve_ar_command() asks Wazuh for agent OS; selects Linux script name or Windows .cmd. 5. wazuh_client.run_active_response() asks Manager to run the command on that agent. 6. Optional Shuffle workflow notification. 7. Audit log written (actor, portal, IP, agent, incident). 8. On the endpoint, .cmd reads JSON from STDIN and launches .ps1. 9. .ps1 applies firewall quarantine; logs QUARANTINE ACTIVE applied=true on success. 10. UI should show Dispatched until verification policy is satisfied — not a fake Verified from “agent online.”

Unisolate reverses state using saved prior firewall settings / markers under ProgramData.

E.4 Windows file pack (detailed)

Source of truth directory:

/opt/mssp-control/deploy/wazuh-active-response/windows/

File	Responsibility
mssp-isolate-host.cmd	AR entrypoint; STDIN JSON; launches PowerShell
mssp-isolate-host.ps1	Quarantine engine (netsh, allow-list, blocks, state files)
mssp-kill-process.cmd / .ps1	Terminate target PID/process
mssp-block-hash.cmd / .ps1	Hash response helper (limited enforcement today)
Install-MsspWindowsEdrAr.ps1	Install onto a Windows host
Test-MsspQuarantineProof.ps1	Prove quarantine effect
../mssp-windows-edr-ar-remediate.zip	Distributable pack

API image copy (keep synced):

backend-api/app/endpoint_configs/windows-edr-ar/

Sync command:

```
./scripts/kb091_sync_windows_edr_ar_pack.sh
```

Register command names on Manager:

```
./scripts/kb090_register_windows_edr_ar_commands.sh
```

E.5 Linux Active Response

Scripts without extensions under deploy/wazuh-active-response/:

- mssp-isolate-host
- mssp-kill-process

- mssp-block-hash

Deploy helper:

```
WAZUH_LINUX_AGENT_HOST=<ip> ./scripts/kb083_deploy_wazuh_edr_ar.sh
```

(Requires a live Linux agent host — VM 105 is currently gone.)

E.6 Forensics collection (expanded)

When COLLECT_FORENSICS runs:

1. Control plane creates edr_forensic_artifacts (awaiting_upload). 2. Issues HMAC-signed upload URL (secret from FORENSICS_SIGNING_SECRET or JWT secret fallback). 3. Collector uploads ZIP with HTTP PUT. 4. API reads request.stream() and writes chunks to disk or S3 multipart — does not load entire file into RAM (prevents OOM). 5. Optional complete callback stores size/SHA256. 6. UI offers short-lived download URL.

Default storage path: /var/lib/mssp/forensics (override EDR_FORENSICS_STORAGE_PATH).

E.7 Background sweeper (expanded)

File: backend-api/app/services/edr_sweeper.py

Setting	Default	Meaning
EDR_SWEEP_INTERVAL	60 seconds	How often to look for stuck rows
EDR_STUCK_TIMEOUT	120 seconds	How old executing may be before timeout

Stuck actions become timeout with message Action timed out (sweeper).

E.8 Engineering lessons (story form)

Lesson 1 — False “verified”

Symptom: UI said isolated; gateway ping and RDP still worked. Cause: “Manager accepted AR” + “agent online” treated as success. Agent online is expected. Fix: Honesty model + endpoint proof line + proof script.

Lesson 2 — AR never ran

Symptom: Dashboard said sent; firewall unchanged. Cause: Windows .cmd deadlocked on more or broke JSON on argv. Fix: STDIN → PowerShell contract documented in the .cmd files.

Lesson 3 — Wrong OS command

Symptom: Windows host got Linux AR names. Cause: OS detection defaulted to Linux. Fix: Unknown OS fails closed; Windows commands use .cmd defaults.

Lesson 4 — PowerShell parse failures

Symptom: Script exits immediately. Cause: Unicode punctuation and \$name: drive-qualified parsing under Windows PowerShell 5.1. Fix: ASCII-safe scripts, UTF-8 BOM, \${name} form.

Lesson 5 — Admin UI gap

Symptom: Customer could isolate; Admin full incident page missing controls. Fix: Admin incident containment UI restored; analysts included in write roles.

Honest open gaps remain in docs/KB091_ENTERPRISE_CONTAINMENT_HONESTY_GAPS.md.

Part F — Command cookbook (fully elaborated)

Unless noted, run on VM 100 as a user that can talk to Docker, from:

```
cd /opt/mssp-control
```

F.1 First morning checks

```
docker compose ps
curl -fsS http://localhost:8000/health | jq .
curl -fsS -o /dev/null -w 'admin:%{http_code}\n' http://localhost:3000/
curl -fsS -o /dev/null -w 'customer:%{http_code}\n' http://localhost:3001/
```

Interpret:

- Health JSON must show database/redis ok.
- Portal codes should be 200.
- A wrong password login should be 401 through /api/auth/login, never 502.

F.2 Rebuild the whole control plane

```
cd /opt/mssp-control
docker compose up -d --build
```

Use after pulling new code that touches API and both frontends.

F.3 Rebuild after API-only changes (avoid 502)

```
cd /opt/mssp-control
docker compose up -d --build backend-api frontend-admin frontend-customer
```

F.4 Auth / API regression

```
./scripts/kb011_validate_protected_apis.sh
```

Run whenever auth, nginx proxy, or protected routes change.

F.5 User management / portal auth checks

```
./scripts/kb088_validate_user_management.sh
# optional deeper:
python3 scripts/validate_user_management.py
```

F.6 EDR validators

```
./scripts/kb083_validate_edr_mxdr.sh
./scripts/kb084_validate_edr_lifecycle_gaps.sh
./scripts/kb090_validate_windows_edr_ar_packaging.sh
./scripts/kb091_validate_edr_containment_honesty.sh
```

F.7 List search / pagination validator

```
./scripts/kb091_validate_list_pagination_search.sh
```

F.8 Onboard a new customer (operator checklist)

1. Admin :3000 → create tenant (unique short code, deployment mode, entitlements). 2. Provision engine bindings (Wazuh group / TheHive tags) — KB-072 path. 3. Create first Customer Administrator (customer_admin). 4. Confirm that user can log into :3001 and cannot log into :3000. 5. Issue fresh per-tenant agent package/token (never reuse another tenant's ZIP). 6. Install agent on endpoint(s). 7. For Windows: confirm Sysmon/4688 telemetry; install AR pack; register AR on Manager. 8. Generate a test alert; confirm it appears under the correct tenant only.

Linux agent install pattern

```
curl -fsSL 'http://192.168.0.201:8000/v1/agent-install/<SHORT_CODE>/<TOKEN>/linux.sh' | sudo bash
```

Windows

Use the tenant package from Admin/Customer package APIs. MSI-style enrollment embeds Manager and group via `agent_package_builder.py`.

F.9 Windows AR maintenance

```
# After editing scripts in deploy/.../windows/
./scripts/kb091_sync_windows_edr_ar_pack.sh

# Ensure Manager ossec.conf knows command names
./scripts/kb090_register_windows_edr_ar_commands.sh
```

On the Windows host (elevated), run installer / proof scripts from the zip as documented in the pack.

F.10 Vulnerability operations

```
./scripts/kb079_run_vuln_scans.sh          # kick scans on 109 (as designed by script)
./scripts/kb079_pull_nuclei_findings.sh    # pull Nuclei → control plane
./scripts/kb079_pull_vuls_findings.sh      # pull Vuls → control plane
./scripts/kb070_pull_greenbone_findings.sh # pull Greenbone CE → control plane
./scripts/kb078_validate_nuclei_vuls_free_stack.sh
./scripts/kb079_validate_nuclei_vuls_integration.sh
```

F.11 TheHive / Shuffle operations

```
export THEHIVE_ORG=MSSP
./scripts/kb061_sync_thehive_alerts.sh
./scripts/kb061_run_periodic_sync.sh
./scripts/kb062_shuffle_control_plane_hop_helper.sh
./scripts/kb061_validate_thehive_control_plane_sync.sh
```

F.12 Database pool & Redis (what to know)

Pool implementation: `backend-api/app/db/session.py`

Env var	Default	Meaning
DB_POOL_MIN_SIZE	5	Warm connections
DB_POOL_MAX_SIZE	20	Cap under load
DB_POOL_TIMEOUT	30	Seconds to wait for a connection

Redis requires password (`REDIS_PASSWORD`). Health endpoint includes Redis status.

F.13 Safe change workflow (team discipline)

1. Inspect live files / git status. 2. Plan; get approval for large changes. 3. Implement only the needed blast radius. 4. Run the matching validator(s). 5. Smoke both portals + the feature path. 6. Commit only when a human asks; never commit `.env`.

Part G — Product reference (portals, tables, APIs) — expanded

G.1 Admin portal map (:3000)

Area	What you do there
Dashboard	KPI tiles, EDR metrics strip, SOC efficiency, incident workspace
Tenants / Customers	Onboard and manage organizations
Users	MSSP staff accounts
Customer Users (under customer)	That tenant's Administrators/Viewers
Appliances	Collectors, activation tokens, health
Assets	Protected assets
Alerts	Triage queue + detail
Incidents	Cases + EDR containment
Vulnerabilities	Scanner findings triage
Recommendations	Customer action items
Notifications	Outbound notice tracking
Reports	Monthly reporting
Audit	Who did what (searchable + detail page)

G.2 Customer portal map (:3001)

Area	What the customer does
Dashboard	Their KPIs and safe overview
Alerts / Incidents	Their visible security events and cases
Assets / Appliances	What is protected / collector health (safe fields)
Recommendations	Actions their team should take
Reports	Published reports
Services	Entitlement-oriented service view
Vulnerabilities	If entitled — customer-safe
Users	Administrators manage viewers/admins
Audit	Tenant-scoped accountability
Account	Profile / password

Dashboard tile destinations (after correction)

Tile	Destination
Active Incidents	/incidents?status=open
Events Collected / monitored	/alerts
Security Alerts	/alerts?severity=high
Collector health (Admin)	/appliances
Open recommendations (Customer)	/recommendations

G.3 Database tables (expanded meanings)

Table	Plain-English meaning
tenants	Customer companies
platform_users	Every portal login (staff + customer)
tenant_contacts	Contact records
appliance_activation_tokens	One-time appliance onboarding secrets
appliances	Customer/site collectors
protected_assets	Things being protected
appliance_heartbeats	Liveness history
security_alerts	Normalized detections
incidents	Working cases
incident_alerts	Links alerts into incidents
incident_timeline / incident_comments	Case narrative
notification_events	Notice history
customer_recommendations	"Please do X" items
monthly_reports	Formal reports
audit_logs	Accountability trail
vulnerabilities	Vuln findings
tenant_entitlements	What services are enabled
tenant_engine_bindings	How tenant maps into Wazuh/TheHive
service_upgrade_requests	Customer asked to upgrade a capability
edr_action_executions	Isolate/kill/forensics attempts
edr_endpoint_isolation	Current quarantine state per agent
edr_telemetry_stats	Aggregate EDR counters
edr_process_events	Process tree fuel
edr_forensic_artifacts	Evidence package metadata
tenant_asset_service_coverage	Which assets get which services
tenant_agent_install_tokens	Agent install tokens

Schema roots: postgres/init/001_mssp_core_schema.sql and migrations 002–020.

G.4 EDR HTTP API cheat sheet

Prefix: /v1/edr

Method	Path	Purpose
GET	/telemetry/process-tree	Process tree data
POST	/actions/execute	Start an action
POST	/actions/callback	Report action result
GET	/actions/{execution_id}	Poll one action
PUT	/forensics/upload/{artifact_id}	Stream upload
GET	/forensics/download/{artifact_id}	Timed download

Method	Path	Purpose
POST	/forensics/complete	Finalize metadata
GET	/incidents/deep-dive	Deep incident EDR context
GET	/metrics/summary	Dashboard strip metrics

Part H — Honesty register (expanded)

H.1 Safe to operate daily

- Dual portals with portal login separation
- Multi-tenant isolation (customer 404)
- Live engines: Wazuh, Suricata, TheHive, Shuffle, Nuclei, Vuls, Greenbone CE
- Windows quarantine AR with proof methodology
- Admin + Customer containment UI
- Audit enrichment for EDR
- List search/pagination on major lists
- Streaming forensics + DB pool + EDR sweeper
- Dashboard KPI links corrected

H.2 Do not sell as finished

Topic	Current reality
Automatic Verified isolation	Needs reliable endpoint callback Wave 1
Block hash	Not full WDAC/AppLocker/ASR
Zeek	Scripts exist; not confirmed live
MISP / Velociraptor / monitoring VM	Not installed
Greenbone Enterprise	Deferred on purpose
Linux endpoint lab	Destroyed; rebuild when needed
Shared callback keys	Must harden before broad self-serve customer isolate

Track containment gaps in docs/KB091_ENTERPRISE_CONTAINMENT_HONESTY_GAPS.md.

Part I — Evolution timeline (compressed)

For the full narrative, challenges, and wiring story, read Part S.

Phase	KB / markers	What it gave you
Foundation	KB-001–009A	Compose, core schema, AI/agent rules
Auth & protected APIs	KB-010–017	Roles, JWT, tenants/users/appliances
Admin UI	KB-018–020	SOC portal foundation

Phase	KB / markers	What it gave you
Customer UI	KB-021–035	Full customer list/detail product surface
Architecture roadmap docs	KB-036–060	Deployment models, future VM plan
Live SIEM / IDS / IR	KB-041–049, 061–063	Wazuh 4.14.6, Suricata, TheHive+Shuffle, ingress
Entitlements & vulns	KB-068–079	Greenbone CE, Nuclei+Vuls, contract onboard
EDR / MXDR	KB-083–084	Actions, lifecycle, forensics, process trees
Windows readiness	KB-088–091	Portal auth, telemetry, quarantine honesty, validators
Ops UX	recent work on main	Audit detail, list search, KPI link fixes
This bible	v2→v3	Operator-first, expanded, live-verified

Large commit that packaged Windows quarantine + ops UX + first blueprint export: d972b93.

Appendix R — Regenerate PDF and DOCX

From VM 100:

```
cd /opt/mssp-control
docker run --rm -v /opt/mssp-control:/work -w /work python:3.12-slim \
  bash -c 'pip install -q python-docx reportlab && python scripts/export_mssp_master_blueprint.py'
```

Outputs:

- /opt/mssp-control/DOCS/MSSP_PLATFORM_MASTER_BLUEPRINT.pdf
- /opt/mssp-control/DOCS/MSSP_PLATFORM_MASTER_BLUEPRINT.docx
- Mirrored copies under docs/

Appendix S — Critical path index on disk

Path	Why it matters
/opt/mssp-control/CONTEXT.md	Short current snapshot
/opt/mssp-control/AGENTS.md	Working rules (some “latest KB” lines may lag git)
/opt/mssp-control/docker-compose.yml	VM 100 runtime
/opt/mssp-control/ansible/inventory/hosts.yml	VM IP inventory
/opt/mssp-control/backend-api/	Product API
/opt/mssp-control/frontend-admin/	Admin portal source
/opt/mssp-control/frontend-customer/	Customer portal source
/opt/mssp-control/postgres/init/	Schema + migrations

Path	Why it matters
/opt/mssp-control/deploy/wazuh-active-response/	AR scripts + Windows zip
/opt/mssp-control/scripts/	Validators and ops helpers
/opt/mssp-control/docs/	KB corpus + this bible

Addenda 1–17 — Operator runbooks, catalogs, and indexes

These chapters complete the book for day-2 operations, sales language, audits, and emergencies.

Addendum 1 — Troubleshooting playbook

Work top-down: portal → API health → database/redis → engine reachability → agent → script.

1.1 Login returns HTTP 502 (Admin or Customer)

Check	Action	
API up?	<code>`curl -fsS http://localhost:8000/health \</code>	<code>jq .`</code> on VM 100
Containers	docker compose ps — all five should be Up	
Classic cause	API recreated; nginx still pointing at old Docker IP	
Fix	docker compose up -d --build backend-api frontend-admin frontend-customer	
Confirm	Bad password via /api/auth/login returns 401, not 502	

1.2 Login returns 403 “wrong portal”

| Meaning | Staff account used on :3001, or customer account used on :3000 |

| Fix | Use the portal that matches the role (Part C) |

1.3 Health says database or redis not ok

Check	Action
Postgres	docker compose ps + logs docker logs mssp-postgres --tail 100
Redis password	Must match .env REDIS_PASSWORD
Disk full	df -h on VM 100

1.4 Alerts missing for a tenant

1. Confirm agent Active on Wazuh Manager (VM 101). 2. Confirm agent group / tenant_engine_bindings for that short code. 3. Remember: fail-closed — no binding ⇒ no silent DEMO attach. 4. Check instant ingress / SOC sync paths and API logs. 5. Confirm Customer portal only shows customer_visible rows.

1.5 Process tree empty on Windows

1. Agent online? 2. Sysmon installed + baseline applied? 3. Security 4688 + command-line auditing on? 4. ossec.conf localfile channels present? 5. Re-run scripts/bootstrap_windows_telemetry.ps1 (elevated) or redeploy tenant Windows package. 6. Confirm edr_process_events receiving rows (SOC/API), not just agent “green.”

1.6 Isolate clicked but host still reachable

1. UI status: prefer Dispatched until proven — do not trust “agent online.” 2. On host, read Active Response log for QUARANTINE ACTIVE applied=true (or FAILED). 3. Confirm Windows AR pack installed; Manager has commands registered (kb090_register_windows_edr_ar_commands.sh). 4. Confirm OS-aware command (Windows .cmd, not Linux name). 5. Run Test-MsspQuarantineProof.ps1. 6. If GPO overrides firewall, treat as containment failure and use network/identity compensating controls (KB-091).

1.7 Vuln findings not appearing

1. SSH to VM 109; confirm Nuclei/Vuls/Greenbone still installed. 2. Re-run pull scripts from VM 100 (kb079_pull_*, kb070_pull_greenbone_findings.sh). 3. Confirm vuln sync API key secret mounted. 4. Check Admin Vulnerabilities page filters/tenant scope.

1.8 TheHive sync quiet

1. TheHive :9000 up on 192.168.0.212? 2. Export THEHIVE_ORG=MSSP (avoid accidental MSSP-Lab default in older script). 3. Run ./scripts/kb061_sync_thehive_alerts.sh and read output/log.

1.9 Customer sees another tenant's data (severest)

1. Stop. Preserve logs. 2. Capture the exact URL, user email, role, timestamp. 3. Query audit_logs and API access logs. 4. This is a P1 security incident — follow Addendum 17. 5. Expected design: wrong tenant → 404. Any cross-tenant leak is a defect.

Addendum 2 — Day-in-the-life walkthroughs

2.1 SOC morning (Admin :3000)

1. Open http://192.168.0.201:3000 → sign in as SOC manager/analyst. 2. Dashboard: note Active Incidents, high alerts, collector health, EDR metrics strip. 3. Open Incidents (search/filter). Pick highest severity. 4. Review process tree / alert detail. 5. If containment needed: Isolate → expect Dispatched → verify on host / proof script → document in incident. 6. Create/update recommendation for customer if they must act. 7. Check Audit for your isolate action (you + portal + agent). 8. Optional: pull vulns / review Vulnerabilities promotions.

2.2 Customer morning (Customer :3001)

1. Open http://192.168.0.201:3001 as Customer Administrator. 2. Dashboard KPIs (events → alerts, recommendations tile). 3. Review open incidents (plain English). 4. If co-managed isolate is required and policy allows: use incident containment (Administrator only — not Viewer). 5. Work Recommendations list. 6. Download latest published report if present. 7. Users page: invite a Viewer if needed.

2.3 Full isolate → proof → unisolate (Windows)

1. Confirm target is the intended agent (Alpha Windows 192.168.0.214 in lab). 2. From Admin incident: Isolate. 3. Note execution id / Dispatched state. 4. On Windows: confirm log QUARANTINE ACTIVE applied=true; RDP/SMB/general LAN should fail; Manager path may still work. 5. Run proof script. 6. Unisolate → confirm restore. 7. Open Audit detail: actor, role, portal, source IP, agent, incident.

2.4 Onboard one new customer (happy path)

1. Admin → create tenant (short code, deployment mode, entitlements). 2. Provision engine bindings (Wazuh group / TheHive tags). 3. Create Customer Administrator. 4. Verify portal separation (customer fails on :3000). 5. Issue fresh agent package/token (never reuse another tenant's ZIP). 6. Install agent + Windows telemetry + AR pack. 7. Register AR on Manager if first Windows fleet / after pack update. 8. Generate test alert; confirm only that tenant sees it. 9. Publish a draft report when ready.

Addendum 3 — Port and firewall matrix

Allow only what you need between VMs. Values are the live on-prem design.

Source	Destination	Port	Purpose
Operator browser	192.168.0.201	3000/tcp	Admin portal
Customer browser	192.168.0.201	3001/tcp	Customer portal
Browser / scripts	192.168.0.201	8000/tcp	API (prefer via /api proxy in prod browsing)
VM 100 API	192.168.0.211	55000/tcp	Wazuh API (AR, agent OS)
Endpoints / sensors	192.168.0.211	1514/tcp	Wazuh agent events
Endpoints	192.168.0.211	1515/tcp	Agent enrollment
Engineers (optional)	192.168.0.211	443/tcp	Wazuh Dashboard
VM 100	192.168.0.212	9000/tcp	TheHive API/UI
Engineers / Shuffle UI	192.168.0.212	3001/tcp	Shuffle (not Customer portal)
VM 100 pullers	192.168.0.219	22/tcp	SSH to vuln host
Operators	192.168.0.219	443/tcp	Greenbone GSA
VM 112 Ansible	101/102/106/109/...	22/tcp	Config automation
Isolated Windows host	192.168.0.211	(manager allow)	Must remain reachable to Manager during quarantine

If blocked: portals work but isolate fails (no 55000); agents go disconnected (no 1514); vulns stop syncing (no SSH to 109); cases stop (no 9000).

Addendum 4 — Secrets and access map (no secret values)

4.1 Where secrets live (never commit)

Location	Examples of contents
/opt/mssp-control/.env	Postgres/Redis/JWT/API port — gitignored
/opt/mssp-control/.secrets/	Files mounted into API: Wazuh API user/password, TheHive password, SOC sync key, ingress token, Shuffle webhook URL, vuln sync key
Wazuh Manager host (root-only paths)	Installer-generated Wazuh creds (KB-041 custody)
Greenbone host-local	GSA admin password (not in Git)
Endpoint mssp-ar.env (optional)	WAZUH_MANAGER_IP=... override

4.2 Who should access what

Role / person	VM 100 SSH	Admin :3000	Customer :3001	Engine UIs (Wazuh/TheHive/GSA)	Ansible 112
Platform admin	Yes (limited)	Yes	No (wrong portal)	Break-glass only	As needed
SOC manager/analyst	Rarely	Yes	Support via Admin tools	Rarely	No
Customer Administrator	No	No	Yes (own tenant)	Never	No
Customer Viewer	No	No	Yes read-only	Never	No
Automation	Keys from 112	No	No	Via playbooks	Yes

4.3 Rotation habits

- Rotate JWT only with planned re-login of all users.
- Rotate appliance API keys via Admin credential rotate endpoint (KB-017).
- Rotate vuln/SOC sync keys in .secrets/ then recreate API container.
- Never paste secrets into chat, KB docs, or git commits.

Addendum 5 — Backup and recovery

Align with docs/KB060_BACKUP_MONITORING_UPGRADE_OPERATIONS_RUNBOOK.md.

5.1 What to back up

Asset	Method
VM 100 whole box	Proxmox snapshot before upgrades
PostgreSQL data	Docker volume postgres_data + logical dump during maintenance
Redis	Less critical (cache); AOF enabled in Compose
.env + .secrets/	Offline encrypted store (not in Git)
Engine VMs 101/102/106/109	Proxmox snapshots + engine-native backups where applicable

Asset	Method
Forensics disk	EDR_FORENSICS_STORAGE_PATH files + DB metadata

5.2 Restore order (control plane)

1. Restore/start VM 100 networking. 2. Restore .env / .secrets/ from secure store. 3. docker compose up -d. 4. Confirm /health. 5. Confirm both portals. 6. Confirm Wazuh API reachability from API container. 7. Do not reuse old per-tenant agent ZIPs after tenant rebuild — issue fresh packages.

5.3 Snapshot discipline

Validate → commit (human) → tag → Proxmox snapshot. Never snapshot only after a failed half-migrate.

Addendum 6 — Printable checklists

6.1 After every control-plane deploy

- ☐ docker compose ps healthy
- ☐ /health ok
- ☐ Admin :3000 loads
- ☐ Customer :3001 loads
- ☐ Bad login → 401 not 502
- ☐ Feature path you changed works
- ☐ Ran relevant kb0NN_validate_*.sh

6.2 New customer onboard

- ☐ Tenant created (unique short code)
- ☐ Deployment mode + entitlements set
- ☐ Engine bindings provisioned
- ☐ Customer Administrator created
- ☐ Portal separation tested
- ☐ Fresh agent package issued
- ☐ Agent online in correct group
- ☐ Windows telemetry + AR (if Windows)
- ☐ Test alert tenant-scoped
- ☐ Customer sees only their data

6.3 Windows host EDR-ready

- ☐ Wazuh agent active
- ☐ Sysmon baseline
- ☐ 4688 + cmdline audit
- ☐ ossec localfiles
- ☐ AR pack installed

- [] Manager AR commands registered
- [] Process tree non-empty after activity
- [] Isolate proof on approved test host

6.4 Before marketing “isolation works”

- [] Live proof: quarantine applied=true
- [] LAN probes fail; Manager path OK
- [] Unisolate restores
- [] Audit row complete
- [] KB-091 Wave-1 gaps understood (callbacks/hash)

Addendum 7 — Glossary

Term	Plain meaning
Tenant	One customer organization in tenants
short_code	Short unique tenant code used in URLs/packages/groups
Control plane	VM 100 product (API + DB + portals)
Engine / adapter	Backend tool (Wazuh etc.) feeding the control plane
Portal-separated login	Admin vs Customer door enforced at login
Customer Administrator	UI name for role customer_admin
Viewer	UI name for role customer_viewer
Fail-closed	If mapping missing, do not guess a tenant
Co-managed	SOC and customer admin can both act (with audit)
Active Response (AR)	Manager tells agent to run a script now
Dispatched	Command sent; effect not yet proven
Verified	Endpoint-proven success (strict bar)
Quarantine / isolate	Default-deny network containment with Manager allow-list
Entitlement	Which paid/enabled capabilities a tenant has
customer_visible	Flag/fields safe to show on Customer portal
SOC sync	Integration pipe between cases/engines and control plane
Appliance	Collector/site device registered to a tenant
Process tree	Parent/child process view for investigation

Addendum 8 — Service catalog (what you sell vs what engine sits behind it)

Use capability language with customers; engines stay internal.

Customer-facing capability	Typical engine behind it	Control-plane outcome
Endpoint / log monitoring	Wazuh + agents	Alerts, assets, health
Network monitoring	Suricata (+ future Zeek)	Network alerts
Incident response / cases	TheHive + portals	Incidents, timelines
Security automation	Shuffle	Auto-ticket / workflows
Vulnerability assessment	Nuclei + Vuls (+ Greenbone CE backup)	Vulnerabilities → recommendations
Endpoint response / containment	Wazuh AR + EDR APIs	Isolate/kill/forensics
Reporting	Control plane exporters	Monthly PDF/Excel
User governance	Control plane RBAC	Admin/Customer user mgmt

Entitlements mapping code: `backend-api/app/services/customer_safe_labels.py` (e.g. Wazuh → “Endpoint monitoring”).

Upgrade requests: KB-076 service upgrade flow.

Addendum 9 — Customer-safe vs SOC-only

Data / capability	SOC Admin portal	Customer portal
Raw Wazuh/Suricata JSON / <code>raw_event</code>	Yes (as needed)	No
Internal notes	Yes	No
MITRE / deep technical internals	Often yes	Only if explicitly approved
Source IPs / many network IPs	Yes	No (forbidden field set)
Engine product names	Yes internally	No — capability labels
Password hashes / API keys / tokens	Metadata only; never raw hashes to UI	No
Cross-tenant view	Staff roles yes	Never
Isolate / kill	SOC write roles	Customer Administrator only
Audit of containment	Yes	Tenant-scoped yes
Draft reports	Yes	No (published/archived only)

Wrong tenant on customer APIs → 404.

Addendum 10 — Audit story (isolate accountability)

Why it exists

When a customer asks “who isolated our server?”, you must answer from platform records, not chat memory.

What a good EDR audit row carries

- Actor user id / email

- Actor role (customer_admin, soc_analyst, ...)
- Portal (customer_portal / mssp_admin_portal)
- Action (isolate / unisolate / kill / ...)
- Target agent / host
- Related incident
- Source IP of the operator
- Timestamp
- Status / detail

How to use it operationally

1. Admin or Customer → Audit → search by action/time/user. 2. Open Audit detail page. 3. Cross-check edr_action_executions status (Dispatched/timeout/verified). 4. If disputed, pull endpoint Active Response log line for the same window.

Co-managed isolate without this trail is commercially unsafe — that is why enrichment was built.

Addendum 11 — Knowledge Base (KB) index

Open the matching file under /opt/mssp-control/docs/.

Topic	Start here
AI/agent rules workflow	KB009
Auth/RBAC	KB010
Protected APIs / 404 tenant	KB011
Route modularization	KB012
Tenants / users / appliances	KB013–017
Admin frontend	KB018–020
Customer portal features	KB021–035
Architecture roadmap	KB036
Cluster/mode planning	KB037–038
Ansible / Wazuh install	KB039–041
Agents / Suricata / Zeek plans	KB042–047
TheHive / Shuffle / workflow	KB047–049, KB061–063
MISP / Greenbone / Velociraptor plans	KB050–055
Admin triage / ingest / template	KB056–058
Ops runbook	KB060
Admin onboarding/ops/reports	KB065–067
Greenbone + vuln path	KB068–070, KB077–079
Entitlements / onboard / upgrades	KB071–076
Alert taxonomy	KB082
EDR/MXDR + lifecycle	KB083–084

Topic	Start here
Lab reset / RBAC audit notes	KB085
User mgmt / portal auth / Windows telemetry	KB088
Containment honesty	KB091
E2E milestone	KB064
This bible	MSSP_PLATFORM_MASTER_BLUEPRINT.md

Addendum 12 — Validator script index

Run from /opt/mssp-control. Prefer the newest script for the area you changed; use kb011 after auth/API/nginx changes.

After you change...	Run
Auth / protected APIs / nginx proxy	./scripts/kb011_validate_protected_apis.sh
User management / portal auth	./scripts/kb088_validate_user_management.sh
Windows telemetry packaging	./scripts/kb088_validate_windows_telemetry_onboarding.sh
EDR APIs	./scripts/kb083_validate_edr_mxdr.sh
EDR lifecycle/forensics	./scripts/kb084_validate_edr_lifecycle_gaps.sh
Windows AR packaging	./scripts/kb090_validate_windows_edr_ar_packaging.sh
Containment honesty rules	./scripts/kb091_validate_edr_containment_honesty.sh
List search/pagination	./scripts/kb091_validate_list_pagination_search.sh
Nuclei/Vuls stack	./scripts/kb078_validate_nuclei_vuls_free_stack.sh
Vuln integration	./scripts/kb079_validate_nuclei_vuls_integration.sh
Greenbone puller	./scripts/kb070_validate_greenbone_live_puller.sh
TheHive sync	./scripts/kb061_validate_thehive_control_plane_sync.sh
Architecture docs	./scripts/kb036_validate_mssp_platform_architecture_roadmap.sh
Enterprise Greenbone plan docs	./scripts/kb077_validate_greenbone_enterprise_readiness_plan.sh

Dozens more historical kb0NN_validate_*.sh exist for earlier modules — use when touching that module.

Addendum 13 — Known quirks (read once, save hours)

1. Two different “port 3001”s: Customer portal on VM 100; Shuffle UI on VM 102. Always include the IP. 2. customer_admin vs Administrator: same role; UI vs DB naming. 3. THEHIVE_DEFAULT_ORG=MSSP vs script default MSSP-Lab: set THEHIVE_ORG when syncing. 4. Agent online during isolate is normal — not proof of quarantine. 5. Do not reuse tenant agent ZIPs across customers. 6. Recreate both frontends with API or risk login 502. 7. KB-036/042 VM tables can be stale — trust CONTEXT.md, inventory, and this bible. 8. VM 105 is gone — old Linux agent notes may reference ghosts. 9. Zeek scripts ≠ Zeek live until verified on VM 106. 10. Block-hash is limited — do not sell as full WDAC. 11. Cursor sandbox/bwrap issues on this VM can break agent shells — use unsandboxed execution when needed (ops note for builders). 12. Git tags vs AGENTS “latest KB-035” — git/HEAD wins.

Addendum 14 — Bible version changelog

Version	What changed
1.0	First master blueprint export (dense engineering draft)
2.0	Operator rewrite; live VM catalog; customer_admin clarification; command cookbook
3.0	Every core section expanded
4.0	Part S journey story, challenges/fixes, tool jobs, wiring, locked decisions
5.0	Addenda 1–17 (troubleshoot, walkthroughs, ports, secrets, backup, checklists, glossary, catalog, safety matrix, audit story, KB/validator indexes, quirks, changelog, UI tour, E2E evidence, platform IR)

Addendum 15 — UI tour (annotated, screenshot placeholders)

You can paste screenshots under each heading in an internal slide deck; the book text stands alone.

15.1 Admin portal (:3000)

Screen	What to notice
Login	Staff only; wrong portal rejected
Dashboard	KPI tiles (correct links), EDR metrics strip (summary), incident table + detail panel
Tenants/Customers	Onboarding, entitlements, deployment mode badges
Users	MSSP staff; customer users under customer context
Alerts / Incidents	Search/pagination; isolate on incident detail
Vulnerabilities	Promote to recommendations
Audit / Audit detail	Accountability drill-down
Appliances / Assets	Collector and asset posture

15.2 Customer portal (:3001)

Screen	What to notice
Login	Customer roles only
Dashboard	Safe KPIs; recommendations tile; no engine brands
Incidents	Plain English; Administrator containment controls
Alerts / Assets / Reports	Customer-visible fields only
Recommendations	Action list for their team
Users	Administrator vs Viewer labels
Audit	Their tenant's actions only
Account	Profile / password

Addendum 16 — Sample E2E evidence (what “good” looks like)

Use as a template when recording proofs. Replace IDs with the run you capture.

16.1 Windows quarantine proof (lab pattern)

Step	Expected signal
Pre-check	Host RDP/WinRM reachable from SOC jump path
Isolate from Admin incident	API returns execution; UI Dispatched
Host log	QUARANTINE ACTIVE applied=true
Effect	Outbound profile Block; LAN probes fail; Manager 192.168.0.211 still reachable
Audit	Row shows actor + mssp_admin_portal + agent + incident
Unisolate	Connectivity restored; quarantine marker cleared

Reference hosts used in build-out: WIN-BL72S84GDTF / 192.168.0.214 / tenant Alpha-Win-Corp path.

16.2 Wazuh → Shuffle → TheHive

Step	Expected signal
Test rule / integration fire	Wazuh integrations log hit
Shuffle workflow	Run success
TheHive	Alert/case visible in org MSSP (or configured org)
Control plane	Incident/alert visible after ingress/sync

16.3 Vuln pull

Step	Expected signal
Pull script	synced=N style success
Admin Vulnerabilities	New rows for tenant
Customer	Sees recommendation only after SOC promotion / visibility rules — not raw scanner dump

Store dated evidence (screenshots, redacted logs) outside Git if they contain customer data.

Addendum 17 — Platform incident response plan

This is IR for your platform, not customer malware IR.

17.1 Severity

Sev	Examples	Response target
P1	Cross-tenant data leak; auth bypass; ransomware on VM 100/101; secret dump in Git	Immediate page; stop bleed
P2	Portals down; Wazuh Manager down; mass agent disconnect	Urgent restore
P3	Single tenant sync broken; one scanner pull failing	Same business day
P4	Cosmetic UI; docs drift	Planned

17.2 P1 first hour

1. Contain: disable public exposure if needed; rotate compromised keys; take volatile notes. 2. Preserve: snapshot VMs; export relevant audit_logs; do not wipe evidence. 3. Scope: which tenants, which portals, which secrets. 4. Communicate: internal owners first; customers if their data was affected (factual, no speculation). 5. Eradicate/recover: patch, rotate, restore from known-good snapshot, re-validate kb011 + health. 6. Post-incident: written timeline; update KB-091 / this bible quirks; add validator if a gap allowed it.

17.3 Suspected cross-tenant leak

1. Disable affected API routes only if necessary (prefer fix-forward with validation). 2. Identify queries missing tenant_id filter. 3. Notify impacted customers per policy. 4. Add regression test to the relevant validate script before closing.

17.4 Compromised .env / .secrets

1. Rotate all listed secrets (Postgres, Redis, JWT, sync keys, Wazuh API, TheHive, vuln key). 2. Recreate containers. 3. Force re-login (JWT). 4. Review audit_logs for abuse window. 5. Confirm .gitignore still excludes secrets.

17.5 Compromised endpoint AR abuse

1. Treat isolate/kill storm as hostile. 2. Freeze Customer Administrator isolate entitlement if needed (policy). 3. Pull audit rows for portal/source IP. 4. Rotate callback/sync keys if forgery suspected (KB-091 shared-key gap).

17.6 Who leads

Area	Typical owner
Control plane / Git / secrets	Platform admin
Wazuh / agents / AR	SOC + platform
Customer comms	Named account owner
Proxmox / network	Infra owner

Keep contact names in an offline runbook (not necessarily in Git).

End of Kestrel Cyber MSSP Platform Operations Bible — version 5.0 (addenda 1–17 included).